AIQA Orchestrator - Deep Backend Audit

Scope: `services/orchestrator` (4,073 lines of Rust, 15 modules), its 3 SQL migrations, and the 5 executor projects it drives. **Date:** 15 August 2026 **Method:** Full read of every module, then live probing against a running orchestrator with real HTTP calls and real runs. Every defect below was **reproduced**, not inferred. Where a hypothesis failed to reproduce, that is stated. **Backend was not modified during this audit.** All findings are observations; no fixes were applied.

---

1. Executive summary

The orchestrator is well-architected and, in its core scheduling logic, correct. The dependency DAG, continue-on-failure semantics, policy gate, auth model and executor sandboxing are all genuinely well-built, and the module boundaries are clean - `scheduler.rs` is pure and side-effect-free, which is why its behaviour is directly testable.

The defects are not in the logic. They are in the seams between the logic and its durability.

The headline finding is a chain of three interacting defects that produces **silent, permanent data corruption under ordinary load**:

1. SQLite runs in default rollback-journal mode with no busy timeout, so concurrent writes fail with `SQLITE_BUSY`.
2. Every persistence failure in `engine.rs` is swallowed - logged at `warn!`/`error!`, then execution continues.
3. The startup recovery pass then mislabels the damaged runs, in one case marking a fully-passed run as `FAILED`.

The result, demonstrated live: a 500-task run that completed with 500 passes is permanently stored as 498 passed + 2 stuck `RUNNING`, and a 50-task run where **every task passed** is permanently stored as `FAILED`.

A second, independent finding: the intelligence layer's history query has **no tenant predicate**, so one project's failure history silently contaminates another's risk scores - directly contradicting the project isolation `auth.rs` enforces everywhere else.

Severity	Count
Critical	2
High	4
Medium	5
Low	4

---

2. What was verified as working

Stated first, because most of this system is sound and the findings below should not be read as a verdict on the whole.

Behaviour	Evidence
Continue-on-failure (SRD §25)	12-task run: t2/t7 failed, other 8 executed
Transitive dependency blocking	`contract` FAILED -> `perf` BLOCKED -> `deploy` BLOCKED, two levels deep
Product vs infrastructure failure split	`FAILED` and `INFRA_ERROR` kept distinct end-to-end
Auth boundary	401 no key - 401 wrong key - 200 valid key; failures written to `audit_log`
Role enforcement	`require_role` server-side on every privileged route
Project scoping (runs/events/evidence)	`require_project_access` enforced, including on the WS handshake
SQL injection	**None.** Every statement is parameterised; the only `format!` builds the connection string
Command injection	`sanitize_token` rejects metacharacters and `..`; covered by 2 regression tests

Behaviour	Evidence
Cycle / unknown-prerequisite rejection	`validate_plan` rejects both
Evidence path traversal	`resolve()` canonicalises and re-checks containment
Event ordering	1,002 events over 500 tasks, zero sequence gaps (in memory)
Real executor invocation	pytest ran as a real subprocess; its genuine traceback captured as evidence

`cargo check --all-targets` clean; `cargo test` 12/12 passing.

---

3. Critical findings

C1 - Persistence failures are swallowed, producing permanent silent corruption

Files: `store.rs:50-62` (no WAL, no busy timeout) - `engine.rs:105-107, 303-308, 356-361, 389-391`

SQLite is opened with only `create_if_missing` and `foreign_keys`:

```
let opts = SqliteConnectOptions::from_str(&format!("sqlite://{}", db_path.display()))?
    .create_if_missing(true)
    .foreign_keys(true);
let pool = SqlitePoolOptions::new().max_connections(8).connect_with(opts).await?;
```

There is **no** `journal_mode(Wal)` and **no** `busy_timeout`. The default rollback journal takes an exclusive lock for every write, and without a busy timeout a blocked writer fails immediately rather than waiting. With `max_connections(8)` and a scheduler that writes concurrently from many task futures, collisions are inevitable.

Every write site then discards the error:

```
if let Err(e) = store.set_task_status(...).await {
    tracing::warn!("failed to persist task completion for {}/{tid}: {e}");
}
```

Execution continues. In-memory state stays correct; the database silently diverges. Because `GET /runs/:id` and `/events` serve from memory while a run is resident, **the damage is invisible until the process restarts.**

Reproduced. 7 `database is locked` incidents during ordinary load testing:

```
WARN failed to persist task completion for 8eec5961.../x244: (code: 5) database is locked
WARN failed to persist event 5a689db1... for run 7cede438...: (code: 5) database is locked
ERROR failed to persist run completion for a0071d45...: (code: 5) database is locked
```

Then, the same two runs read before and after an orchestrator restart:

Run	Before restart (memory)	After restart (database)
`8eec5961` (500 tasks)	COMPLETED - 500 PASSED - 1002 events	COMPLETED - **498 PASSED + 2 RUNNING** - **1001 events**
`a0071d45` (50 tasks)	**COMPLETED** - 50 PASSED - 102 events	**FAILED** - 50 PASSED - 103 events

Two tasks that passed are permanently recorded as still running. One event is permanently gone from a log that SRD §5 requires to be complete and gap-free.

Why it matters beyond the data: the audit log, the metrics endpoint, the intelligence layer and every post-hoc report read from this database. A quality system whose own record of what happened is wrong is worse than one that admits it does not know.

Fix direction: enable WAL and a busy timeout (`.journal_mode(Journal::Wal).busy_timeout(Duration::from_secs(5))`); then treat a persistence failure as a real error - retry, or mark the run degraded - rather than logging and continuing.

C2 - Startup recovery marks fully-passed runs as FAILED

File: `engine.rs:215-246`, specifically line 229

```
let final_state = if touched_any { RunState::Partial } else { RunState::Failed };
```

`recover_incomplete_runs` finds runs left non-terminal by an unclean shutdown. If it had to force any task terminal, the run becomes `Partial` - reasonable. But if it touched **nothing** - meaning every task was already terminal and only the *run-level* completion write was lost - it marks the run `Failed`.

That is precisely the C1 scenario, and it is exactly backwards: a run whose tasks all passed is the best possible evidence it did not fail.

Reproduced. Run `a0071d45`: 50 tasks, all 50 `PASSED` in the database, run state `FAILED`. The startup log confirms the path: `recovered 5 run(s) left non-terminal by a prior unclean shutdown`.

Fix direction: derive the run state from its tasks' actual statuses, exactly as the normal completion path does at `engine.rs:379-386`, rather than from whether recovery had to intervene.

---

4. High findings

H1 - Intelligence history has no tenant predicate: cross-project data leak

Files: `store.rs:569-579` - called from `intelligence.rs:154`

```
pub async fn task_history_by_name(&self, name: &str, limit: i64) -> Result<Vec<TaskHistoryRow>> {
    sqlx::query("SELECT t.status, t.finished_at FROM tasks t
                 WHERE t.name = ? AND t.finished_at IS NOT NULL
                 ORDER BY t.finished_at DESC LIMIT ?")
```

The query filters on task **name only**. `intelligence.rs` has `pid` in scope and passes it to `upsert_quality_node`, but not here - so the *node* is per-project while the *history that scores it* is global.

Reproduced with two project-scoped operator keys and one shared task name:

```
alpha ran 'shared-task-name' 3x -> all FAILED
beta  ran 'shared-task-name' 1x -> PASSED

alpha: failure_rate=1.00  window=3/3  risk=0.400
beta:  failure_rate=0.75  window=3/4  risk=0.367   <-- beta never failed
```

Project `beta` sees a 75% failure rate over a 4-run window, having run the task once, successfully. Those three failures belong to `alpha`.

This is a **confidentiality issue as well as a correctness one**: it discloses the existence and failure behaviour of another tenant's tests, and it contradicts the isolation `require_project_access` enforces on every other route.

Fix direction: add `project_id` to the query and to the `tasks` table (it currently only exists on `runs`, so this needs a join or a denormalised column), plus an index on `(name, project_id)` - see M4.

H2 - `started_at` is recorded before a task can run, corrupting all timing data

File: `engine.rs:290-297`

```
let newly_eligible = {
    let mut engine = record.engine.write().await;
    let ids = engine.eligible();
    for id in &ids { engine.mark_started(id); }   // status=Running, started_at=now, for ALL
    ids
};
```

Every eligible task is marked started - status `RUNNING`, `started_at = now` - in one batch. The semaphore permit that actually admits it to execution is not acquired until line 309, inside the dispatch loop. When eligible tasks outnumber `concurrency`, tasks sit waiting while already timestamped as running.

Reproduced. Five tasks, 600 ms each, `concurrency: 1`:

```
task   started_at offset   reported duration
s1     +      0ms          2528ms
s2     +      0ms          2628ms
s3     +      0ms          2697ms
s4     +      0ms          2766ms
s5     +      0ms          3134ms

start-time spread across 5 serialized tasks: 0ms   (expected ~2400ms)
```

All five claim the same start instant, and s5 reports 3,134 ms for 600 ms of work - a 5x overstatement. `TASK_STARTED` events are emitted at the same wrong moment.

Consequential damage: `intelligence.rs:175-178` computes `execution_cost_ms` from exactly these timestamps, and `cost_norm` carries **15% of the risk score weight**. So queue-wait time is being laundered into risk scores. Any duration or timeline view built on this data is misleading.

Fix direction: acquire the permit first, then `mark_started`, then emit `TASK_STARTED`.

H3 - No timeout on any executor subprocess

File: `executor.rs:266, 285, 314, 335, 365`

Every adapter awaits `.output()` with no bound:

```
match shell_command("npx", &args, &self.project_dir).output().await { ... }
```

A hung Playwright browser, a Maestro call with no device attached, or a Schemathesis run against an unresponsive target blocks forever. That future holds its semaphore permit permanently, so the run's concurrency budget shrinks and the scheduler loop never observes the completion - `in_flight` never decrements. **One hung tool can stall a run indefinitely.**

There is no cancellation token, no bounded retry, and no run-level deadline anywhere in the codebase. SRD §20 requires all three.

Related: `TaskStatus::Timeout` is declared in `domain.rs` and is part of the FRD §2 ten-status model, but `grep` confirms it is produced **only** by `SimulatedExecutor`. No real executor can ever emit it - the status is unreachable in production.

Fix direction: wrap each `.output()` in `tokio::time::timeout` with a per-task configurable budget, mapping expiry to `TaskStatus::Timeout`.

H4 - `POST /runs` fails at ~100 tasks

File: `store.rs:66-113`

`insert_run` issues one awaited `INSERT` per task and one per prerequisite, with **no transaction and no batching**. In default journal mode each is its own implicit transaction with its own fsync.

Reproduced (single request each, clean client):

Tasks	Result	Latency	Per task
10	HTTP 201	0.77 s	77 ms
50	HTTP 201	8.58 s	172 ms
100	**HTTP 500**	14.28 s	-
250	**HTTP 500**	15.92 s	-

The request blocks for the entire insert, so a 50-task plan holds an HTTP connection for 8.6 seconds. Beyond ~100 tasks it fails outright, the 500s coinciding with the `database is locked` errors in C1.

For a platform whose PRD targets enterprise regression suites, a hard ceiling near 100 tasks per run is a fundamental limit.

Fix direction: wrap `insert_run` in a single transaction and batch the inserts; enabling WAL (C1) compounds the improvement.

---

5. Medium findings

M1 - `insert_run` is not atomic

`store.rs:66-113`. No transaction wraps the run + task + dependency inserts. A failure partway through leaves a run row with an incomplete task set and no rollback. Same fix as H4.

M2 - SSRF via executor URL parameters

`executor.rs:309, 363`. `pytest`'s `base_url` and `schemathesis`'s `url` come straight from the `POST /runs` body. `sanitize_token` permits `:` and `/`, so `http://169.254.169.254/...` or any internal host passes. Schemathesis will then actively fuzz that target from the orchestrator's network position. Requires an operator key, but operator is not intended to be a network-pivot role. **Fix:** allow-list schemes and hosts, or require targets to be pre-registered.

M3 - No bound on plan size

`policy.rs:65`. Policy checks `concurrency` against `max_concurrency` but never the task count. A plan with 100,000 tasks is accepted by validation and then attempts 100,000 sequential inserts. **Fix:** add `AIQA_MAX_TASKS_PER_RUN` to `PolicyConfig`.

M4 - Missing indexes on two hot paths

`tasks.name` has no index, yet `task_history_by_name` runs once per terminal task per run and does a full table scan. `api_keys.key_hash` is `UNIQUE` (so implicitly indexed - fine), but `tasks(name)` and `tasks(run_id, status)` are not.

M5 - A lagging WebSocket client silently misses events

`api.rs:586`. On `RecvError::Lagged(_)` the handler does `continue`, dropping the skipped events with no notification. The client is never told to resynchronise. Since the broadcast channel holds 1,024 events and a 500-task run emits 1,002, a slow client on a large run is realistic. SRD §5's at-least-once guarantee is not met. **Fix:** on lag, close with a distinct code or push a resync marker so the client refetches from `?since=`.

---

6. Low findings

L1 - Recovery uses a magic sequence number

`engine.rs:231`: `let sequence = tasks.len() as u64 + 1000;`. A run with more than 1,000 events - a 500-task run produces 1,002 - can collide with a real sequence. Observed live: run `a0071d45` gained an event (102 -> 103) from this path.

L2 - `kind` is unsanitised in evidence filenames

`evidence.rs:37-45` filters `task_id` but interpolates `kind` directly. Currently always the literal `"log"`, so not exploitable - latent if another caller is added.

L3 - 10% of the risk score is permanently dead weight

`intelligence.rs:186-195`. `security_sensitivity`, `business_criticality` and `change_frequency` are hardcoded `0.0`, so the maximum achievable `risk_score` is 0.90. Honestly documented in the code and in the persisted `inputs` JSON, but the score is not on the scale it appears to be.

L4 - Approve/reject are process-local

`api.rs:270-282`. Both return 404 if the run's `RunRecord` is not in this process's memory, so a restart voids every pending approval. Documented in the code; a real constraint on multi-process deployment.

---

7. Hypotheses that did NOT reproduce

Recorded because a clean negative is a result.

- ? **SQLite contention at moderate load.** 40 tasks at concurrency 8 completed with 82/82 events, zero gaps, zero warnings. Contention only appeared at 100+ task plans with several runs resident (C1). The initial "any concurrency will break it" framing was wrong.
- ? **SQL injection.** Every statement parameterised; no dynamic SQL construction anywhere.
- ? **Command injection through executor params.** `sanitize_token` rejected every metacharacter payload tested, and the rejection is covered by an existing regression test.
- ? **Evidence path traversal.** `resolve()` blocks `..` and re-checks containment after canonicalisation.

A first measurement of `POST /runs` latency was discarded: the probe called `urlopen` twice per request, inflating every figure. §4/H4 uses corrected single-request numbers.

---

8. Code health

Metric	Value
Rust source	4,073 lines, 15 modules
Largest module	`store.rs` (1,015)
Unit tests	12, all passing
Tested modules	`scheduler`, `policy`, `executor`, `intelligence`
Untested modules	`store`, `engine`, `api`, `auth`, `evidence`, `metrics`, `webhook`
`unsafe`	1 (a Windows `CommandExt` import, not a memory-safety escape)
`.unwrap()` / `.expect()`	10 / 4
`panic!`	0

The test coverage gap tracks the defect distribution exactly. Every finding in this report is in an untested module - `engine.rs` (C1, C2, H2), `store.rs` (C1, H1, H4, M1), `executor.rs` (H3, M2), `api.rs` (M5). The four tested modules produced no findings above Low. That is not a coincidence, and it is the most actionable single fact here.

Two `.expect()` calls are on the startup path (`store.connect`, `evidence::new`) where failing fast is correct. `semaphore.acquire_owned().await.expect("semaphore open")` in `engine.rs:309` is safe only because the semaphore is never closed.

---

9. Recommended order of work

1. **Enable WAL + busy timeout** (`store.rs`) - one-line change, removes the trigger for C1 and most of H4.
2. **Stop swallowing persistence errors** (`engine.rs`) - the corruption in C1 is caused by continuing, not by the failed write.
3. **Fix recovery's state derivation** (`engine.rs:229`) - compute run state from task statuses.
4. **Scope the intelligence history query by project** (`store.rs:569`) - closes the H1 tenant leak.
5. **Acquire the permit before** `mark_started` (`engine.rs:290`) - fixes timing, and with it `execution_cost_ms` and risk scoring.

6. **Add executor timeouts** (`executor.rs`) - makes `TaskStatus::Timeout` reachable and stops a hung tool stalling a run.
7. **Transaction + batching in** `insert_run` (`store.rs`) - raises the H4 ceiling.
8. **Tests for** `engine.rs` **and** `store.rs` - the two modules carrying every critical finding.

Items 1-3 are small and together eliminate both Critical findings.

---

10. Verdict

The architecture is sound; the durability layer is not yet trustworthy.

The scheduling core does what the PRD claims, and the parts under test are correct. But a quality-engineering platform's central promise is that its record of what happened is accurate, and this one will - under load that it accepts without complaint - permanently record passing runs as failed, leave completed tasks marked running, drop events from a log specified to be gap-free, and let one tenant's failures move another tenant's risk scores.

None of that is visible while the process stays up. All of it survives a restart.

The good news is proportionate: the two Critical findings share a single root cause and three small changes close them. The core scheduler, which is the hard part and the part hardest to retrofit, needs nothing.

Every command output quoted here was executed against a live orchestrator during this audit. Hypotheses that failed to reproduce are recorded in §7. No backend file was modified.